NAGARDOCS AI

Intelligent Government Document Digitization System

Hack-The-Gap 2.0 | Problem Statement SG2

Flutter • FastAPI • OpenAI GPT • EasyOCR • Supabase

1. PRODUCT REQUIREMENTS DOCUMENT

1.1 Problem Statement (SG2)

Municipal corporations across India manage millions of physical government documents — marksheets, birth certificates, property records, income certificates, ration cards — entirely through manual paper-based processes. This results in slow retrieval, data loss, no searchability, and zero analytics. NagarDocs AI solves this with one scan.

1.2 Solution — What NagarDocs AI Does

- Teacher / staff scans or uploads any government document (PDF or image)
- EasyOCR extracts raw text from the document
- ChatGPT API classifies document type and extracts structured fields automatically
- Document saved to searchable digital cabinet (like File Explorer)
- Staff can search any record instantly by name, ID, or document type
- Analytics dashboard shows total records, types, languages, tamper alerts
- Export to Excel for any document category in one click

1.3 Core Features

Feature	P	Description
Universal Document Upload	P0	Accept any govt document — PDF, JPG, PNG
EasyOCR Text Extraction	P0	Extract raw text from scanned physical documents
ChatGPT Auto-Classification	P0	Classify doc type + extract fields automatically
Digital Cabinet (File Explorer)	P0	Organize documents into folders like Windows Explorer
AI Auto-Sort	P0	One click — AI creates folders and sorts all documents
Instant Search	P0	Search by name, ID, date, document type
Tamper Detection	P0	ChatGPT flags potentially altered documents
Confidence Score	P0	Shows extraction accuracy 0-100%
Analytics Dashboard	P0	Total docs, types breakdown, language stats
Excel Export	P1	Export any cabinet to .xlsx
Multi-language Support	P1	English, Hindi, Marathi documents
Marksheet SGPA Analytics	P1	Full CGPA/SGPA dashboard (existing feature)

1.4 Tech Stack

Layer	Technology	Why
Mobile App	Flutter	Cross-platform, live demo on phone
State Management	Riverpod	Clean provider pattern
Backend	FastAPI (Python)	Fast async, ML-friendly

Layer	Technology	Why
OCR Engine	EasyOCR	Free, works offline, Hindi/Marathi support
AI Classification	OpenAI GPT-4o-mini	Classifies ANY doc type, extracts fields, tamper check
Database	Supabase (PostgreSQL)	Auth + DB + Storage
File Storage	Supabase Storage	Store uploaded PDFs + images
Excel Export	pandas + openpyxl	Professional Excel output
Deployment	ngrok (demo) / Railway (prod)	Live accessible URL

1.5 Winning Demo Flow (24-hour hackathon)

Step	What Judge Sees
1	Open Flutter app on live phone
2	Upload a municipal marksheet photo
3	App shows: Marksheet detected (confidence 94%)
4	Extracted: Name, PRN, SGPA, Program, Semester
5	Upload a birth certificate photo
6	App shows: Birth Certificate detected (confidence 91%)
7	Extracted: Name, DOB, Registration No, Issuing Authority
8	Click Auto-Sort — folders appear automatically
9	Search Sarika — instant result across all documents
10	View Analytics — 5 docs, 3 types, 0 tamper alerts
11	Export Marksheets folder to Excel — downloads instantly

2. FLUTTER APP — COMPLETE FOLDER STRUCTURE

2.1 Top Level

```
lib/
  main.dart                      ← ProviderScope, Supabase.initialize
  app.dart                       ← MaterialApp.router, dark theme
  router.dart                    ← GoRouter all routes
  core/                          ← Constants, network, utils
  features/                      ← One folder per feature
  shared/                        ← Reusable widgets + providers
```

2.2 Core Layer

```
core/
  constants/
    app_colors.dart              ← Navy/teal dark palette
    app_text_styles.dart         ← h1/h2/body/caption
    app_strings.dart             ← All UI strings
    api_endpoints.dart           ← All backend URL constants
  errors/
    failure.dart                 ← Sealed Failure class
  network/
    dio_client.dart              ← Dio + base URL + timeout
    auth_interceptor.dart        ← Auto-attach JWT token
  storage/
    secure_storage.dart          ← flutter_secure_storage wrapper
  utils/
    gpa_utils.dart               ← SGPA/CGPA computation
    file_utils.dart              ← File size, extension helpers
    validators.dart              ← Form validation
```

2.3 Feature: Auth

```
features/auth/
  data/auth_repository.dart      ← Supabase login/signup
  domain/teacher_model.dart      ← Teacher(id, email, name, college)
  presentation/
    screens/login_screen.dart
    screens/signup_screen.dart
    widgets/auth_input_field.dart
    widgets/auth_submit_button.dart
    providers/auth_provider.dart ← StateNotifier<AuthState>
    providers/auth_state.dart    ← initial/loading/authenticated/error
```

2.4 Feature: Home Dashboard

```
features/home/
  presentation/
    screens/home_screen.dart     ← 4 stat cards + recent docs + quick actions
    widgets/
      stat_overview_card.dart    ← Total docs / Types / Tamper alerts
      recent_doc_list.dart       ← Last 5 uploaded documents
      quick_action_row.dart      ← Upload / Search / Sort buttons
```

```
      doc_type_chart.dart        ← Pie chart: doc types breakdown
```

2.5 Feature: Document Upload (CORE)

```
features/upload/
  data/
    upload_repository.dart       ← POST /upload/document
    upload_dto.dart              ← UploadResponse, ProcessedDocument
  domain/
    document_model.dart          ← Document(id, type, fields, confidence)
    extracted_fields_model.dart  ← name, date, id_number, other_fields
  presentation/
    screens/upload_screen.dart   ← Camera + gallery + PDF picker
    widgets/
      upload_drop_zone.dart      ← Tap area with camera/gallery icons
      processing_overlay.dart    ← Animated OCR + AI processing screen
      result_card.dart           ← Shows extracted fields after processing
      confidence_badge.dart      ← Green/yellow/red confidence score
      tamper_alert_banner.dart   ← Red banner if tamper detected
      doc_type_chip.dart         ← Marksheet / Birth Cert / Property etc
    providers/
      upload_provider.dart       ← StateNotifier<UploadState>
      upload_state.dart          ← idle/picking/processing/done/error
```

2.6 Feature: Digital Cabinet (File Explorer)

```
features/cabinet/
  data/
    cabinet_repository.dart      ← CRUD /cabinets
    cabinet_dto.dart             ← CabinetCreate, CabinetResponse
  domain/
    cabinet_model.dart           ← Cabinet(id, name, type, color, doc_count)
  presentation/
    screens/
      cabinet_list_screen.dart   ← Grid of folder cards — like File Explorer
      cabinet_detail_screen.dart ← Documents inside a folder
      document_detail_screen.dart← Full extracted fields + raw text
    widgets/
      cabinet_folder_card.dart   ← Folder icon + name + doc count
      auto_sort_button.dart      ← One-tap AI auto-sort with animation
      create_cabinet_dialog.dart ← Name + color picker
      document_list_tile.dart    ← Doc row: type icon + name + date + confidence
      move_to_cabinet_sheet.dart ← Bottom sheet to move doc to another folder
    providers/
      cabinet_provider.dart      ← StateNotifier<CabinetState>
      cabinet_state.dart         ← loading/loaded(cabinets)/error
```

2.7 Feature: Search

```
features/search/
  data/search_repository.dart    ← GET /search?q=...&doc_type=...
  presentation/
    screens/search_screen.dart   ← Search bar + filters + results
    widgets/
      search_bar_widget.dart     ← Animated search bar
      filter_chip_row.dart       ← Filter by doc type chips
```

```
      search_result_tile.dart    ← Result row with highlighted match
      empty_search_view.dart     ← Empty state with suggestions
    providers/search_provider.dart
```

2.8 Feature: Analytics

```
features/analytics/
  data/analytics_repository.dart ← GET /analytics (global) + /analytics/{project_id}
  presentation/
    screens/analytics_screen.dart ← Global dashboard + per-project tabs
    widgets/
      total_docs_card.dart
      doc_type_breakdown_card.dart ← Bar chart per type
      language_stats_card.dart     ← English/Hindi/Marathi counts
      tamper_alert_card.dart       ← Count + list of flagged docs
      sgpa_analytics_card.dart     ← Existing marksheet analytics
      topper_board_card.dart
      pass_fail_card.dart
    providers/analytics_provider.dart
```

2.9 Feature: Export

```
features/export/
  data/export_repository.dart    ← GET /export/{cabinet_id}
  presentation/
    screens/export_screen.dart
    widgets/
      export_cabinet_selector.dart ← Pick which folder to export
      download_button.dart
      share_button.dart
    providers/export_provider.dart
```

2.10 Shared Layer

```
shared/
  widgets/
    app_button.dart              ← Primary/secondary/danger + loading
    app_text_field.dart          ← Consistent input field
    app_loader.dart              ← Shimmer + full-screen loaders
    app_error_view.dart          ← Error + retry
    app_empty_view.dart          ← Empty state illustration
    app_snackbar.dart            ← Success/error/info
    section_card.dart            ← White rounded card wrapper
    stat_chip.dart               ← Colored stat chip
  providers/
    supabase_provider.dart
    dio_provider.dart
    theme_provider.dart
```

2.11 Router — All Routes

Route	Screen
/login	LoginScreen
/signup	SignupScreen

Route	Screen
/home	HomeScreen — dashboard
/upload	UploadScreen — scan/upload doc
/cabinets	CabinetListScreen — file explorer
/cabinets/:id	CabinetDetailScreen — folder contents
/cabinets/:id/doc/:docId	DocumentDetailScreen
/search	SearchScreen
/analytics	AnalyticsScreen
/export	ExportScreen
/projects	ProjectsListScreen — marksheet projects
/projects/:id	ProjectDetailScreen — Upload/Review/Analytics/Export tabs

3. BACKEND — COMPLETE FOLDER STRUCTURE

3.1 Top Level

```
backend/
  main.py                        ← FastAPI app, CORS, all routers
  requirements.txt
  .env                           ← SUPABASE_URL, SUPABASE_KEY, OPENAI_API_KEY
  Dockerfile

  app/
    api/                         ← Route files
    core/                        ← Config, DB, security
    schemas/                     ← Pydantic schemas
    services/                    ← All business logic
```

3.2 API Routes — app/api/

File	Routes
auth.py	POST /auth/signup POST /auth/login
projects.py	GET/POST/PUT/DELETE /projects GET /projects/{id}
upload.py	POST /upload/{project_id} POST /upload/document (universal)
jobs.py	GET /jobs/{id} — poll OCR job status
review.py	GET /review/{project_id} PUT /review/{project_id}/result/{id} POST /confirm
analytics.py	GET /analytics/{project_id} GET /analytics/global
export.py	GET /export/{project_id} GET /export/cabinet/{cabinet_id}
templates.py	GET/POST/DELETE /templates
cabinets.py	GET/POST/PUT/DELETE /cabinets POST /cabinets/auto-sort
search.py	GET /search?q=...&doc_type=...
documents.py	GET /documents GET /documents/{id} DELETE /documents/{id}

3.3 Services — app/services/ (ALL LOGIC HERE)

ocr_service.py

```
# Uses EasyOCR to extract text from PDF / image
# Input:  raw file bytes
# Output: raw text string
# Supports: English, Hindi, Marathi
# pip install easyocr

import easyocr
reader = easyocr.Reader(['en', 'hi'])  # load once at startup

async def extract_text(file_bytes: bytes) -> str:
    result = reader.readtext(file_bytes, detail=0)
```

```
    return ' '.join(result)
```

ai_service.py (THE SECRET WEAPON)

```
# ChatGPT classifies ANY document + extracts fields + detects tamper
# Input:  raw OCR text
# Output: structured JSON dict

async def classify_and_extract(ocr_text: str) -> dict:
    response = openai.chat.completions.create(
        model='gpt-4o-mini',
        messages=[{
            'role': 'user',
            'content': f'''
Extract structured data from this Indian government document.
Return ONLY JSON:
{
  document_type: marksheet|birth_certificate|property_record|
                 ration_card|income_certificate|other,
  confidence_score: 0-100,
  language: english|hindi|marathi|mixed,
  extracted_fields: {name, date, id_number, issuing_authority, other_fields},
  is_tampered: true|false,
  tamper_reason: string or null,
  summary: one line
}
Document: {ocr_text}'''
        }],
        response_format={'type': 'json_object'},
        temperature=0
    )
    return json.loads(response.choices[0].message.content)
```

cabinet_service.py

```
# Auto-sort: reads all unorganized docs → creates folders → assigns docs

async def auto_sort(supabase):
    docs = supabase.table('documents').select('*').is_('cabinet_id', None).execute()
    groups = {}
    for doc in docs.data:
        dtype = doc['document_type']
        groups.setdefault(dtype, []).append(doc['id'])
    for dtype, ids in groups.items():
        cab = supabase.table('cabinets').upsert(
            {'name': dtype.replace('_',' ').title()+'s', 'type': 'auto'},
            on_conflict='name').execute()
        cab_id = cab.data[0]['id']
        supabase.table('documents').update({'cabinet_id': cab_id}).in_('id',
ids).execute()
```

analytics_service.py

```
# Global analytics across ALL documents (not just marksheets)
```

```
# Returns: total_docs, by_type{}, by_language{}, tamper_count, recent[]
```

gpa_service.py (existing — unchanged)

```
# SGPA/CGPA computation for marksheet projects
```

validation_service.py (existing — unchanged)

```
# Flags total mismatch, invalid SGPA, missing fields
```

export_service.py

```
# 2 export modes:
# 1. Project export: Roll|Name|Subject marks|SGPA|CGPA (existing)
# 2. Cabinet export: DocType|Name|Date|IDNumber|Confidence|Filename
```

3.4 Schemas — app/schemas/

```
auth_schema.py          ← SignupRequest, LoginRequest, AuthResponse
project_schema.py       ← ProjectCreate, ProjectResponse (existing)
upload_schema.py        ← UploadJobResponse, ProcessedDocumentResponse
review_schema.py        ← StudentResultResponse, CorrectionRequest (existing)
document_schema.py      ← DocumentCreate, DocumentResponse, ExtractedFields
cabinet_schema.py       ← CabinetCreate, CabinetResponse, AutoSortResponse
search_schema.py        ← SearchResponse, SearchResult
analytics_schema.py     ← GlobalAnalytics, ProjectAnalytics
template_schema.py      ← TemplateCreate, TemplateResponse (existing)
```

3.5 Full API Map

Method	Endpoint	Returns
POST	/auth/signup	JWT token + teacher data
POST	/auth/login	JWT token + teacher data
GET	/projects	List of teacher projects
POST	/projects	New project
GET	/projects/{id}	Single project
DELETE	/projects/{id}	204
POST	/upload/{project_id}	Upload job (marksheet OCR)
POST	/upload/document	Universal doc upload → GPT extract
GET	/jobs/{id}	Job status + progress
GET	/review/{project_id}	All student rows
PUT	/review/{project_id}/result/{id}	Updated row
POST	/review/{project_id}/confirm	Confirm results
GET	/analytics/{project_id}	Marksheet analytics

Method	Endpoint	Returns
GET	/analytics/global	Global doc stats
GET	/export/{project_id}	Excel download (marksheets)
GET	/export/cabinet/{id}	Excel download (any cabinet)
GET	/cabinets	List all cabinets
POST	/cabinets	Create cabinet
PUT	/cabinets/{id}	Rename/recolor cabinet
DELETE	/cabinets/{id}	Delete cabinet
POST	/cabinets/auto-sort	AI auto-sort all documents
GET	/documents	All documents (paginated)
GET	/documents/{id}	Single document detail
DELETE	/documents/{id}	Delete document
GET	/search	Search by name/ID/type
GET	/templates	List templates
POST	/templates	Create template
DELETE	/templates/{id}	Delete template

4. DATABASE SCHEMA (Supabase / PostgreSQL)

4.1 All Tables Overview

Table	Purpose	Key Columns
teachers	Teacher login accounts	id, email, name, college, department
projects	One exam event per class	id, teacher_id, name, program, semester, status
upload_jobs	OCR processing jobs	id, project_id, status, processed, error_message
students	Master student records	prn, seat_no, name, program
student_results	SGPA/CGPA per student per exam	id, project_id, student_prn, sgpa, cgpa, status, rank
subject_marks	Per-subject grade data	id, result_id, course_code, grade_obtained, grade_point, earned_gp
templates	Marksheet column layouts	id, teacher_id, name, columns (JSONB)
documents	Any govt document (NEW)	id, document_type, confidence, extracted_fields, is_tampered
cabinets	Document folders (NEW)	id, name, type (auto/manual), color, icon

4.2 New Tables SQL

documents table

```
CREATE TABLE documents (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  cabinet_id        UUID REFERENCES cabinets(id) ON DELETE SET NULL,
  document_type     TEXT,        -- marksheet|birth_certificate|property_record|...
  confidence        INT,         -- 0-100 from ChatGPT
  language          TEXT,        -- english|hindi|marathi|mixed
  extracted_fields  JSONB,       -- {name, date, id_number, issuing_authority, other}
  is_tampered       BOOLEAN DEFAULT FALSE,
  tamper_reason     TEXT,
  summary           TEXT,        -- one-line GPT summary
  raw_text          TEXT,        -- full OCR output
  filename          TEXT,
  file_url          TEXT,        -- Supabase Storage URL
  created_at        TIMESTAMPTZ DEFAULT NOW()
);

-- Full text search index
CREATE INDEX idx_docs_search ON documents
  USING GIN(to_tsvector('english', raw_text));
CREATE INDEX idx_docs_type ON documents(document_type);
CREATE INDEX idx_docs_cabinet ON documents(cabinet_id);
```

cabinets table

```
CREATE TABLE cabinets (
```

```
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name        TEXT NOT NULL UNIQUE,  -- 'Marksheets', 'Birth Certificates'
  type        TEXT DEFAULT 'manual', -- auto | manual
  color       TEXT DEFAULT '#1A3C5E',
  icon        TEXT DEFAULT 'folder',
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

Existing tables — no changes needed

```
teachers, projects, upload_jobs, students,
student_results, subject_marks, templates
-- all remain exactly as before
```

4.3 RLS Policies for New Tables

```
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE cabinets  ENABLE ROW LEVEL SECURITY;

-- Any authenticated user can manage documents + cabinets
CREATE POLICY 'doc_auth' ON documents
  FOR ALL USING (auth.role() = 'authenticated');

CREATE POLICY 'cabinet_auth' ON cabinets
  FOR ALL USING (auth.role() = 'authenticated');
```

5. DEPENDENCIES

5.1 Flutter pubspec.yaml

Package	Purpose
flutter_riverpod: ^2.5.1	State management
go_router: ^13.2.0	Navigation
dio: ^5.4.3	HTTP client
supabase_flutter: ^2.5.0	Auth + storage
file_picker: ^8.1.2	Pick PDFs
image_picker: ^1.0.7	Camera capture
open_filex: ^4.3.4	Open downloaded Excel
path_provider: ^2.1.3	Device file paths
flutter_secure_storage: ^9.2.2	JWT storage
fl_chart: ^0.68.0	Analytics charts
shimmer: ^3.0.0	Loading skeleton
freezed_annotation: ^2.4.4	Immutable models
json_annotation: ^4.9.0	JSON serialization

5.2 Backend requirements.txt

Package	Purpose
fastapi	Web framework
uvicorn[standard]	ASGI server
pydantic[email] + pydantic-settings	Validation + config
supabase	Database + storage client
openai	ChatGPT API for doc classification
easyocr	OCR — works offline, Hindi/Marathi support
Pillow	Image preprocessing for OCR
pymupdf	PDF → image conversion for OCR
python-multipart	File upload parsing
pandas + openpyxl	Excel export
python-jose[cryptography]	JWT tokens
passlib[bcrypt]	Password hashing
pytest + httpx	Testing

6. 24-HOUR HACKATHON BUILD PLAN

Hours	What to Build	Who
0–3	Flutter: Home + Upload screens UI. Backend: FastAPI running + /upload/document endpoint	All
3–6	Backend: EasyOCR integration. Test OCR on marksheet image. Return raw text	Backend dev
6–9	Backend: ChatGPT AI service. Classify doc + extract fields. Test with 3 doc types	Backend dev
9–12	Flutter: Processing overlay + result card + confidence badge. Full upload flow working	Flutter dev
12–15	Backend: Cabinets CRUD + auto-sort endpoint. Supabase tables created	Backend dev
15–18	Flutter: Cabinet list screen (file explorer UI) + auto-sort button + search screen	Flutter dev
18–20	Flutter: Analytics dashboard + export. Polish all screens. Test full demo flow	All
20–22	Fix bugs from full demo run. Ensure demo works 100% on phone with ngrok	All
22–24	GitHub commits every hour. Prepare 5-page report + PPT slides	All

Key Points for Judges

- Your existing marksheet app = 60% of this already built
- ChatGPT API handles ANY document — no ML training needed in 24 hours
- File Explorer UI is something every municipal officer immediately understands
- Search feature solves the #1 municipal pain point — instant record retrieval
- Live demo on real phone is what wins hackathons